Лабораторная работа 6 «Применение генетических алгоритмов для повышения качества моделей машинного обучения с учителем**»**

В этой лабораторной работе нам понадобится Python 3 и следующие библиотеки:

- `deap`;
- `numpy`;
- `pandas`;
- `matplotlib`;
- `seaborn`;
- `sklearn` – описывается здесь же.

Кроме того, мы будем использовать набор данных *Zoo* (https://archive.ics.uci.edu/ml/datasets/zoo).

Машинное обучение с учителем

Термин **машинное обучение** обычно относится к компьютерной программе, которая получает данные на входе и порождает данные на выходе. Наша цель – обучить эту программу, называемую также **моделью**, порождать правильные выходные данные по имеющимся входным, не программируя это соответствие явно.

В процессе обучения модель учится строить отображение между входами и выходами, корректируя свои внутренние параметры. Один из распространенных способов обучить модель – подать ей на вход данные, для которых известен правильный выход. Для каждого элемента входных данных мы сообщаем модели, каким должен быть выход, поэтому она может настроить себя так, чтобы порождать нужные выходы для всех входов. Вот этот процесс настройки и является сутью обучения.

С годами было разработано много типов моделей машинного обучения. У каждой модели свои внутренние параметры, которые влияют на отображение между входами и выходами, и значения этих параметров можно настраивать, как показано на рисунке ниже.



Настройка параметров модели машинного обучения

Например, модель, реализованная как решающее дерево, могла бы содержать несколько предложений IF-THEN и формулироваться следующим образом:

IF <входное значение> МЕНЬШЕ. ЧЕМ <некоторое пороговое значение>
THEN <перейти к некоторой ветви>

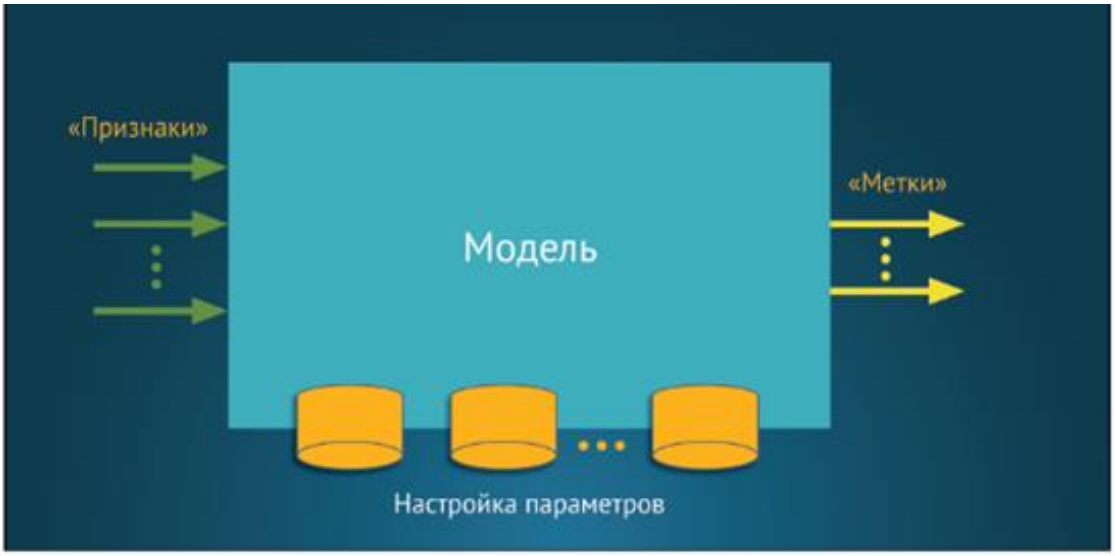
В этом случае пороговое значение и идентификатор целевой ветви – параметры, настраиваемые в процессе обучения.

Для настройки внутренних параметров с каждым типом моделей ассоциирован **алгоритм обучения**, который перебирает входные и выходные данные и стремится установить как можно более точное соответствие между входами и выходами. Для этого типичный алгоритм обучения вычисляет разность (ошибку) между фактическим и желаемым выходом, а затем пытается минимизировать эту ошибку, корректируя внутренние параметры модели.

Два основных типа машинного обучения с учителем – **классификация** и **регрессия** – будут описаны в следующих подразделах.

Классификация

В задаче классификации модель должна определить, к какой категории принадлежит вход. Каждая категория представлена одним выходом (**меткой**), а входные данные называются **признаками**.



Модель классификации в машинном обучении

Например, в хорошо известном наборе данных об ирисах Iris (https://archive.ics.uci.edu/ml/datasets/Iris) признаков четыре: длина лепестка, ширина лепестка, длина чашелистика, ширина чашелистика. Эти характеристики были измерены для реальных цветков ириса.

На выходе возможна одна из трех меток, описывающих сорт ирисов: **Iris setosa**, **Iris virginica** или **Iris versicolor**.

Если на вход подаются данные, представляющие измерения некоторого конкретного цветка, то мы ожидаем, что для правильной метки будет выведено значение **high**, а для двух других – **low**:

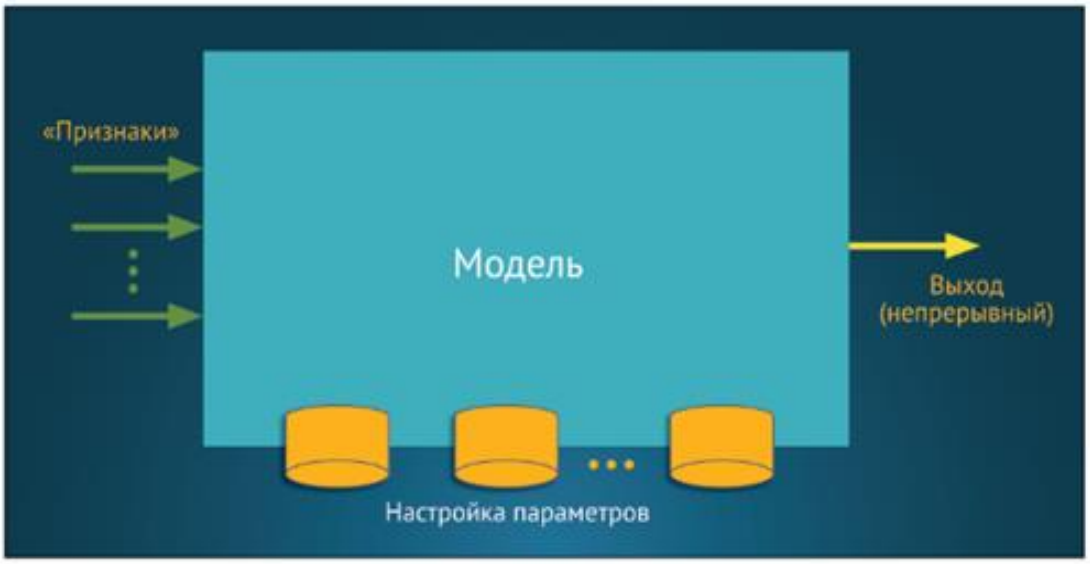


Классификатор ирисов

У задач классификации много практических применений, например одобрение банковских займов и максимального кредита по банковской карте, обнаружение почтового спама, распознавание рукописных цифр и лиц. Ниже в этой главе мы продемонстрируем классификацию животных с помощью набора данных Zoo.

Регрессия

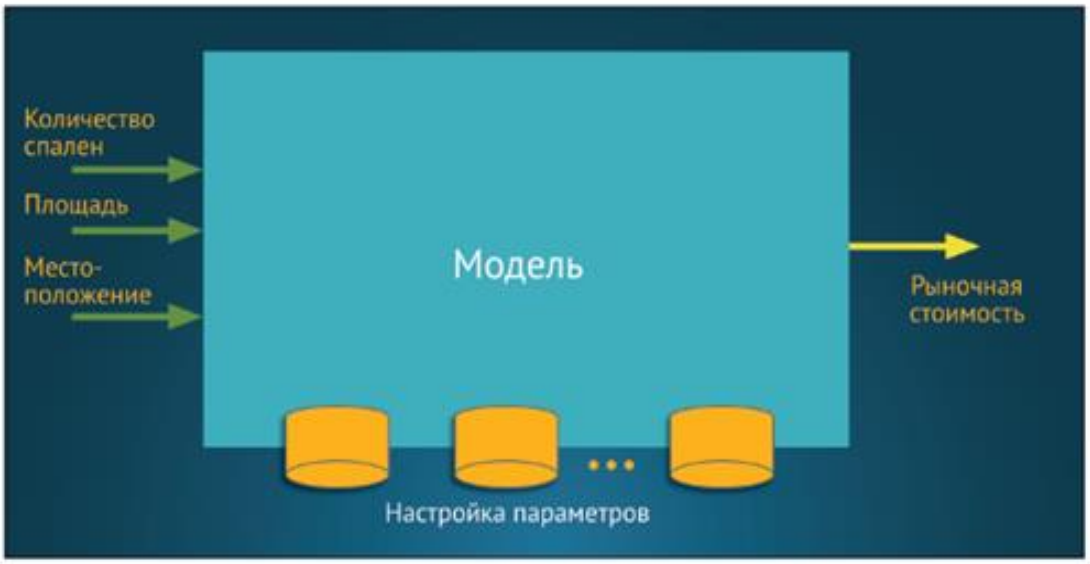
В отличие от классификации, модель регрессии отображает входные значения на единственное выходное значение с непрерывной областью значений.



Модель регрессии в машинном обучении

Ожидается, что модель будет предсказывать правильное выходное значение для поданных на вход данных.

Практические приложения регрессии включают предсказание стоимости ценных бумаг, качества вина или рыночной цены дома, как показано на рисунке ниже.



Регрессионная модель цены дома

Здесь на вход подаются признаки, содержащие информацию о доме, а выходом является предсказанная стоимость дома.

Существует много моделей классификации и регрессии – некоторые из них описаны в следующем подразделе.

Алгоритмы обучения с учителем

Как уже было сказано, у любой модели обучения с учителем имеется набор внутренних настраиваемых параметров и алгоритм, настраивающий их в попытке достичь желаемого результата.

Перечислим некоторые из распространенных моделей (алгоритмов) машинного обучения.

- **Решающие деревья:** семейство алгоритмов, в которых используется древовидный граф, вершины (узлы) которого представляют решения, а ветви – их последствия.
- **Случайные леса:** алгоритмы, которые создают много решающих деревьев на этапе обучения и используют их комбинацию для формирования выхода.
- **Метод опорных векторов:** входные данные рассматриваются как точки в пространстве, а алгоритм стремится разделить их плоскостью, так чтобы зазор между точками, принадлежащими разным категориям, был как можно больше.
- **Искусственные нейронные сети:** модели, состоящие из большого числа простых блоков, называемых нейронами, которые могут быть связаны между собой различными способами. С каждой связью ассоциируется вес, управляющий уровнем сигнала, передаваемого от одного нейрона другому.

Существуют методы, позволяющие повысить качество таких моделей. Один из них – выделение *признаков* – обсуждается в следующем разделе.

Выделение признаков в обучении с учителем

В предыдущем разделе мы видели, что модель обучения с учителем получает множество входов, называемых **признаками**, и отображает их на множество выходов. Предполагается, что информация, содержащаяся в признаках, полезна для определения значений выходов. На первый взгляд кажется, что чем больше информации подается на вход, тем выше шансы на правильное предсказание выходов. Но во многих случаях ситуация прямо противоположна; если некоторые признаки нерелевантны или избыточны, то верность моделей может снизиться (иногда значительно).

Выделение признаков – это процесс отбора самых важных и полезных признаков из общего их множества. Помимо повышения верности модели, успешное выделение признаков дает следующие преимущества:

- уменьшается время обучения модели;
- обученная модель проще, ее легче интерпретировать;

- обученная модель лучше обобщается, т. е. лучше работает на новых данных, которые не предъявлялись в процессе обучения.

Генетические алгоритмы естественно рассматривать при решении задачи о выделении признаков. В следующем разделе мы продемонстрируем их применение на примере искусственно сгенерированного набора данных.

Выделение признаков для задачи регрессии Фридмана-1

В задаче регрессии Фридмана-1, придуманной Фридманом и Брейманом, единственный выход y является следующей функцией пяти входных значений $x_0, \ldots, x_4$ и случайного шума:

$$y(x_0, x_1, x_2, x_3, x_4) = 10 \cdot \sin(\pi \cdot x_0 \cdot x_1) + 20(x_2 - 0.5)^2 + 10x_3 + 5x_4 + noise \cdot N(0,1).$$

Переменные $x_0, \ldots, x_4$ – независимые случайные величины, равномерно распределенные в интервале [0, 1]. Последнее слагаемое в формуле – случайно сгенерированный шум. Шум имеет нормальное распределение и умножен на константу *noise*, определяющую его уровень.

Библиотека scikit-learn (`sklearn`), написанная на Python, предоставляет функцию `make_friedman1()`, которая умеет генерировать набор данных, содержащий любое количество примеров. Каждый пример состоит из случайно сгенерированных значений $x_0, \ldots, x_4$ и соответствующего им значения y. Но вот что интересно – мы можем попросить функцию добавить произвольное число нерелевантных входных переменных к пяти исходным, задав параметр `n_features` больше 5. Так, если положить `n_features = 15`, то будет сгенерирован набор данных, содержащий пять исходных входных переменных (признаков), по которым вычислялось значение y, и еще 10 дополнительных признаков, вообще никак не связанных с выходом. Эту возможность можно использовать, например, для тестирования устойчивости различных моделей регрессии к шуму и присутствию нерелевантных признаков в наборе данных.

Мы можем воспользоваться данной функцией для проверки эффективности генетических алгоритмов как механизма выделения признаков. В нашем тесте функция `make_friedman1()` будет создавать набор данных с 15 признаками, а генетический алгоритм – искать подмножество признаков, при котором качество оказывается наилучшим. Ожидается, что генетический алгоритм отберет первые пять признаков и отбросит остальные в предположении, что верность модели действительно лучше, когда на вход подаются только релевантные признаки. Функция приспособленности генетического алгоритма будет использовать модель регрессии, которая для каждого потенциального решения – подмножества набора признаков – обучается на наборе данных, содержащем только выделенные признаки.

Как обычно, начнем с выбора подходящего представления решения.

Представление решения

Цель нашего алгоритма – найти подмножество признаков, при котором получается наилучшее качество модели. Поэтому в решении должно быть указано, какие признаки отобраны, а какие отброшены. Проще всего представлять индивидуума списком двоичных значений. Каждое значение соответствует одному признаку. Если значение равно 1, то соответствующий признак отобран, если 0 – то нет.

Как мы увидим в следующем подразделе, присутствие 0 в решении означает выбрасывание соответствующего столбца признаков из набора данных.

Представление решения на Python

Для инкапсуляции задачи Фридмана-1 о выделении признаков мы создали Python-класс `Friedman1Test`, который находится в файле по адресу https://github.com/PacktPublishing/Hands-On-Genetic-Algorithms-with-Python/blob/master/Chapter07/friedman.py.

Ниже описаны основные части этого класса.

1. Метод класса `__init__()` создает набор данных:

```
self.X, self.y = datasets.make_friedman1(n_samples=self.numSamples,
                                          n_features=self.numFeatures,
                                          noise=self.NOISE,
                                          random_state=self.randomSeed)
```

2. Затем он разбивает данные на два поднабора – обучающий и тестовый – с помощью метода `model_selection.train_test_split()` из библиотеки scikit-learn:

```
self.X_train, self.X_validation, self.y_train,
self.y_validation = \
    model_selection.train_test_split(self.X, self.y,
test_size=self.VALIDATION_SIZE, random_state=self.randomSeed)
```

   Разделение данных на обучающий и тестовый наборы позволяет обучить модель регрессии на обучающем наборе, в котором имеется правильная метка, а затем протестировать на отдельном тестовом наборе, не предоставляя модели правильных меток, а сравнивая с ними сделанные предсказания. Таким образом, можно проверить, действительно ли модель способна к обобщению, а не просто запомнила обучающие данные.

3. Затем мы создаем модель регрессии типа Gradient Boosting Regressor (регрессия с градиентным усилением – GBR). Эта модель создает ансамбль (или агрегат) решающих деревьев на этапе обучения:

```
self.regressor =
GradientBoostingRegressor(random_state=self.randomSeed)
```

Заметим, что конструктору передается начальное значение генератора случайных чисел, которое будет использоваться моделью. Тем самым мы обеспечиваем воспроизводимость результатов.

4. Метод класса `getMSE()` определяет качество нашей модели регрессии для набора выделенных признаков. Он принимает список двоичных значений – 1 означает, что соответствующий признак отобран, а 0 – что отброшен. Затем метод удаляет те столбцы обучающего и тестового наборов, которые соответствуют отброшенным признакам.

```
zeroIndices = [i for i, n in enumerate(zeroOneList) if n == 0]
currentX_train = np.delete(self.X_train, zeroIndices, 1)
currentX_validation = np.delete(self.X_validation, zeroIndices, 1)
```

5. После этого модифицированный обучающий набор, содержащий только выделенные признаки, используется для обучения модели регрессии, а модифицированный тестовый набор – для оценки ее предсказаний:

```
self.regressor.fit(currentX_train, self.y_train)
prediction = self.regressor.predict(currentX_validation)
return mean_squared_error(self.y_validation, prediction)
```

   Для оценки модели регрессии будем использовать **среднеквадратическую ошибку** (**СКО**), т. е. среднее арифметическое разностей квадратов между фактическими и предсказанными моделью значениями. Чем меньше эта величина, тем лучше модель регрессии.
6. Метод `main()` создает экземпляр класса `Friedman1Test` с 15 признаками. Затем он в цикле вызывает метод `getMSE()`, чтобы оценить качество модели на первых *n* признаках, где *n* увеличивается от 1 до 15:

```
for n in range(1, len(test) + 1):
    nFirstFeatures = [1] * n + [0] * (len(test) - n)
    score = test.getMSE(nFirstFeatures)
```

Выполнение программы показывает, что по мере добавления первых пяти признаков качество повышается. Но каждый последующий признак ухудшает качество модели:

```
1 первых признаков: оценка = 47.553993
2 первых признаков: оценка = 26.121143
3 первых признаков: оценка = 18.509415
4 первых признаков: оценка = 7.322589
5 первых признаков: оценка = 6.702669
6 первых признаков: оценка = 7.677197
7 первых признаков: оценка = 11.614536
8 первых признаков: оценка = 11.294010
9 первых признаков: оценка = 10.858028
10 первых признаков: оценка = 11.602919
11 первых признаков: оценка = 15.017591
12 первых признаков: оценка = 14.258221
13 первых признаков: оценка = 15.274851
```

```
14 первых признаков: оценка = 15.726690
15 первых признаков: оценка = 17.187479
```

Это наглядно видно на графике, показывающем, что минимум СКО достигается, когда берутся первые пять признаков.



График среднеквадратической ошибки для теста Фридмана-1

В следующем разделе мы выясним, сможет ли генетический алгоритм выделить эти первые пять признаков.

Решение с помощью генетического алгоритма

Чтобы выделить лучший набор признаков в нашем тесте регрессии с помощью генетического алгоритма, мы написали на Python программу, которая находится в файле по адресу https://github.com/PacktPublishing/Hands-On-Genetic-Algorithms-with-Python/blob/master/Chapter07/01-solve-friedman.py.

Напомним, что мы представляем хромосому списком нулей и единиц, в котором 1 означает, что список отобран, а 0 – что отброшен. С точки зрения генетического алгоритма, это делает нашу задачу похожей на задачу OneMax или задачу о рюкзаке 0-1, которые мы уже решали. Разница в том, что функция приспособленности теперь возвращает СКО модели регрессии, вычисляемую в классе `Friedman1Test`.

Ниже описаны основные шаги нашего решения.

1. Сначала создаем экземпляр класса `Friedman1Test` с нужными параметрами:

```
friedman = friedman.Friedman1Test(NUM_OF_FEATURES,
NUM_OF_SAMPLES, RANDOM_SEED)
```

2. Поскольку мы стремимся минимизировать СКО модели регрессии, определим единственную цель – минимизирующую стратегию приспособления:

```
creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
```

3. Так как решение представлено списком нулей и единиц, воспользуемся следующими определениями из инструментария для создания начальной популяции:

```
toolbox.register("zeroOrOne", random.randint, 0, 1)
toolbox.register("individualCreator", tools.initRepeat,
creator.Individual, toolbox.zeroOrOne, len(friedman))
toolbox.register("populationCreator", tools.initRepeat, list,
toolbox.individualCreator)
```

4. Затем просим генетический алгоритм использовать метод `getMSE()` объекта класса `Friedman1Test` для вычисления приспособленности:

```
def friedmanTestScore(individual):
    return friedman.getMSE(individual), # вернуть кортеж

toolbox.register("evaluate", friedmanTestScore)
```

5. Применяем турнирный отбор размера 2 и операторы скрещивания и мутации, специализированные для двоичного списка:

```
toolbox.register("select", tools.selTournament, tournsize=2)
toolbox.register("mate", tools.cxTwoPoint)
toolbox.register("mutate", tools.mutFlipBit, indpb=1.0/len(friedman))
```

6. Как и раньше, используем элитистский подход, т. е. без изменения копируем лучших на данный момент индивидуумов из зала славы в следующее поколение:

```
population, logbook = elitism.eaSimpleWithElitism(population, toolbox,
                                                   cxpb=P_CROSSOVER,
                                                   mutpb=P_MUTATION,
                                                   ngen=MAX_GENERATIONS,
                                                   stats=stats,
                                                   halloffame=hof,
                                                   verbose=True)
```

После прогона алгоритма для 30 поколений с размером популяции 30 получается такой результат:

```
-- Лучший индивидуум = [1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
-- Лучшая приспособленность = 6.702668910463287
```

Это показывает, что для обеспечения лучшей СКО (приблизительно 6.7) были отобраны первые пять признаков. Заметим, что генетический алго-

ритм не делал никаких предположений о множестве исследуемых признаков, т. е. не знал, что ищет именно *первые n* признаков. Он просто искал наилучшее подмножество признаков.

В следующем разделе мы перейдем от искусственно сгенерированных данных к реальному набору данных и воспользуемся генетическим алгоритмом, чтобы выделить лучшие признаки для задачи классификации.

Выделение признаков для классификации набора данных Zoo

В репозитории машинного обучения Калифорнийского университета в Ирвайне (https://archive.ics.uci.edu/ml/index.php) хранится 350 наборов данных, доступных научно-исследовательскому сообществу. Эти наборы можно использовать для экспериментов с различными моделями и алгоритмами. Типичный набор, содержащий ряд признаков (входов) и желаемый выход, представлен в форме столбцов с описанием назначения каждого.

В этом разделе мы будем работать с набором данных Zoo (https://archive.ics.uci.edu/ml/datasets/zoo), в котором описано 101 животное с помощью следующих 18 признаков:

№	Признак	Тип данных
1	название животного	уникальное
2	шерсть	Boolean
3	перья	Boolean
4	яйцекладущее	Boolean
5	млекопитающее	Boolean
6	летающее	Boolean
7	водное	Boolean
8	хищник	Boolean
9	зубастое	Boolean
10	наличие позвоночника	Boolean
11	дышащее	Boolean
12	ядовитое	Boolean
13	плавники	Boolean
14	количество ног	Numeric (принимает значения из множества [0,2,4,5,6,8])
15	хвост	Boolean
16	домашнее	Boolean
17	размером с кошку	Boolean
18	тип	Numeric (целое в диапазоне [1,7])

Большинство признаков имеют тип Boolean (принимают значение 1 или 0), указывающий на наличие или отсутствие некоторого атрибута, например шерсти, плавников и т. д. Первый признак, **название животного**, чисто информационный, он не участвует в процессе обучения.

Этот набор данных используется для тестирования алгоритмов классификации, которые должны отобразить входные признаки в две или более категорий (меток). Последний признак, **тип**, как раз и представляет категорию и используется в качестве выхода. Всего в этом наборе семь категорий. Например, тип 5 соответствует категории, в которую входят лягушка, тритон и жаба. Итак, модель классификации, обучаемая на этом наборе, будет использовать признаки со 2 по 17 для предсказания признака 18 (тип животного).

Как и раньше, мы хотим использовать генетический алгоритм для выделения признаков, дающих лучшее предсказание. Начнем с создания Python-класса, представляющего классификатор, обученный на этом наборе данных.

Представление задачи на Python

Чтобы инкапсулировать процесс выделения признаков для классификации набора данных Zoo, мы создали Python-класс `Zoo`. Он находится в файле по адресу https://github.com/PacktPublishing/Hands-On-Genetic-Algorithms-with-Python/blob/master/Chapter07/zoo.py.

Ниже описаны основные части этого класса.

1. Метод класса `__init__()` загружает набор данных Zoo из интернета и пропускает первый признак, название животного:

```
self.data = read_csv(self.DATASET_URL, header=None,
usecols=range(1, 18))
```

2. Затем он разделяет данные на входные признаки (первые 16 из оставшихся столбцов) и выходную категорию (последний столбец):

```
self.X = self.data.iloc[:, 0:16]
self.y = self.data.iloc[:, 16]
```

3. Вместо того чтобы разбивать данные на обучающий и тестовый наборы, как в предыдущем разделе, мы решили воспользоваться k-групповым перекрестным контролем. Это означает, что данные разбиваются на k равных частей и модель прогоняется k раз – в каждом прогоне $k - 1$ частей используются для обучения, а оставшаяся – для тестирования. На Python это легко сделать с помощью метода `model_selection.KFold()` из библиотеки scikit-learn:

```
self.kfold = model_selection.KFold(n_splits=self.NUM_FOLDS,
random_state=self.randomSeed)
```

4. Далее мы создаем модель классификации на основе решающего дерева. В классификаторе такого типа на этапе обучения строится древовидная структура, которая позволяет разбивать набор данных на меньшие части и в конечном итоге выдавать предсказание:

```
self.classifier = DecisionTreeClassifier(random_state=self.randomSeed)
```

Заметим, что конструктору передается начальное значение генератора случайных чисел, которое будет использоваться классификатором. Тем самым мы обеспечиваем воспроизводимость результатов.

5. Метод `getMeanAccuracy()` вычисляет качество классификатора для множества отобранных признаков. Как и метод `getMSE()` класса `Friedman1Test`, он принимает список двоичных значений, в котором 1 соответствует отобранному признаку, а 0 – отброшенному. Затем метод удаляет те столбцы обучающего и тестового наборов, которые соответствуют отброшенным признакам.

```
zeroIndices = [i for i, n in enumerate(zeroOneList) if n == 0]
currentX = self.X.drop(self.X.columns[zeroIndices], axis=1)
```

6. Модифицированный набор данных, содержащий только отобранные признаки, используется для k-группового перекрестного контроля и оценки качества классификатора на отдельных группах. В нашем классе значение k принято равным 5, поэтому каждый раз выполняется пять оцениваний:

```
cv_results = model_selection.cross_val_score(self.classifier,
currentX, self.y, cv=self.kfold, scoring='accuracy')
return cv_results.mean()
```

   Для оценки классификатора применяется метрика, называемая **верностью**, – доля правильно классифицированных примеров. Например, верность 0.85 означает, что 85 % всех примеров были классифицированы правильно. Поскольку мы обучаем и оцениваем классификатор k раз, то используем верность, усредненную по этим k вычислениям.

7. Метод `main()` создает экземпляр класса Zoo и оценивает классификатор, обученный на всех 16 признаках, присутствующих в представлении, содержащем все единицы:

```
allOnes = [1] * len(zoo)
print("-- Выделены все признаки: ", allOnes, ", accuracy = ",
zoo.getMeanAccuracy(allOnes))
```

Результат выполнения метода `main` показывает, что при обучении классификатора методом 5-группового перекрестного контроля на всех 16 признаках достигается верность 91 %:

```
-- Выделены все признаки: [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
accuracy = 0.9099999999999999
```

В следующем подразделе мы попробуем улучшить верность классификатора, выделив только часть признаков. И, как вы догадались, воспользуемся для этого генетическим алгоритмом.

Решение с помощью генетического алгоритма

Для выделения наилучшего набора признаков в задаче классификации набора данных Zoo мы написали на Python программу, которая находится в файле по адресу https://github.com/PacktPublishing/Hands-On-Genetic-Algorithms-with-Python/blob/master/Chapter07/02-solve-zoo.py.

Как было описано в предыдущем разделе, хромосома будет представлена списком нулей и единиц, соответствующих отброшенным и отобранным признакам.

Ниже описаны основные шаги программы.

1. Сначала создаем экземпляр класса Zoo и передаем конструктору начальное значение генератора случайных чисел, чтобы обеспечить воспроизводимость результатов:

```
zoo = zoo.Zoo(RANDOM_SEED)
```

2. Поскольку мы стремимся минимизировать верность модели классификации, определим единственную цель – максимизирующую стратегию приспособления:

```
creator.create("FitnessMax", base.Fitness, weights=(1.0,))
```

3. Как и в предыдущем разделе, воспользуемся следующими определениями из инструментария, чтобы создать начальную популяцию индивидуумов, описываемых списками нулей и единиц:

```
toolbox.register("zeroOrOne", random.randint, 0, 1)
toolbox.register("individualCreator", tools.initRepeat,
creator.Individual, toolbox.zeroOrOne, len(zoo))
toolbox.register("populationCreator", tools.initRepeat, list,
toolbox.individualCreator)
```

4. Далее мы вызываем метод `getMeanAccuracy()` объекта `Zoo` для вычисления приспособленности. Для этого пришлось внести два изменения:
   - мы запретили алгоритму не выбирать ни одного признака (создавать индивидуума, представленного только нулями), поскольку в таком случае классификатор возбудил бы исключение;
   - мы добавили небольшой штраф за каждый отобранный признак, чтобы побудить алгоритм выделять меньше признаков. Штраф совсем невелик (0.001), поэтому будет использоваться, только чтобы разрешить неоднозначность в случае, когда качество двух классификаторов одинаково – тогда предпочтение будет отдано тому, который обошелся меньшим числом признаков.

```
def zooClassificationAccuracy(individual):
    numFeaturesUsed = sum(individual)
    if numFeaturesUsed == 0:
        return 0.0,
    else:
```

```
        accuracy = zoo.getMeanAccuracy(individual)
        return accuracy - FEATURE_PENALTY_FACTOR *
numFeaturesUsed, # вернуть кортеж

toolbox.register("evaluate", zooClassificationAccuracy)
```

5. Применяем турнирный отбор размера 2 и операторы скрещивания и мутации, специализированные для двоичного списка:

```
toolbox.register("select", tools.selTournament, tournsize=2)
toolbox.register("mate", tools.cxTwoPoint)
toolbox.register("mutate", tools.mutFlipBit,indpb=1.0/len(zoo))
```

6. Как и раньше, используем элитистский подход, т. е. без изменения копируем лучших на данный момент индивидуумов из зала славы в следующее поколение:

```
population, logbook = elitism.eaSimpleWithElitism(population,
toolbox, cxpb=P_CROSSOVER, mutpb=P_MUTATION,
ngen=MAX_GENERATIONS, stats=stats, halloffame=hof, verbose=True)
```

7. В конце прогона мы печатаем все содержимое зала славы, чтобы увидеть лучшие найденные алгоритмом результаты. Печатается как приспособленность, включающая штраф за количество признаков, так и фактическая верность:

```
print("- Лучшие решения:")
for i in range(HALL_OF_FAME_SIZE):
    print(i, ": ", hof.items[i], ", приспособленность = ",
hof.items[i].fitness.values[0],
          ", верность = ", zoo.getMeanAccuracy(hof.items[i]),
          ", признаков = ", sum(hof.items[i]))
```

После выполнения алгоритма для 50 поколений с популяцией размера 50 и залом славы размера 5 мы получили:

```
- Лучшие решения:
0 : [0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0] , приспособленность = 0.964 ,
верность = 0.97 , признаков = 6
1 : [0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1] , приспособленность = 0.963 ,
верность = 0.97 , признаков = 7
2 : [0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0] , приспособленность = 0.963 ,
верность = 0.97 , признаков = 7
3 : [1, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0] , приспособленность = 0.963 ,
верность = 0.97 , признаков = 7
4 : [0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, 1, 0] , приспособленность = 0.963 ,
верность = 0.97 , признаков = 7
```

Эти результаты показывают, что все пять лучших решений достигли верности 97 %, используя шесть или семь признаков из 16. Благодаря штрафу за количество признаков самым лучшим является решение с шестью признаками:

- перья;
- млекопитающее;
- летающее;
- наличие позвоночника;
- плавники;
- хвост.

Итак, выделив именно эти шесть признаков из 16, мы смогли не только уменьшить размерность задачи, но и повысить верность модели с 91 % до 97 %. Если вам кажется, что это не такое уж большое достижение, взгляните на него по-другому – как на уменьшение частоты ошибок с 9 % до 3 %, что считается весьма значительным улучшением качества классификации.

РЕЗЮМЕ

В этой лабораторной работе мы познакомились с машинным обучением и двумя основными типами встречающихся в нем задач: регрессией и классификацией. Затем мы узнали о потенциальном положительном влиянии выделения признаков на качество моделей для решения этих задач. Основным содержанием данной лабораторной работы стали два примера применения генетических алгоритмов к повышению качества моделей за счет выделения признаков. В первом случае нам удалось выделить истинные признаки в тестовой задаче регрессии Фридмана-1, а во втором мы выделили самые полезные признаки в наборе данных Zoo для классификации.

Задание для самостоятельной работы.

1. Изучить алгоритм … /Chapter07/01… Получить результаты. Изменить начальные значения ГА перезапустить алгоритм.
2. Изучить алгоритм … /Chapter07/02… Получить результаты. Изменить начальные значения ГА перезапустить алгоритм.